

InternSpark Internship Program

TASK 2

Database Integration & Migrations

Candidate : Vasavi Thellajeera

Candidate ID : IS-2026-10396

Domain : Java Developer Intern

Mentor : Akash Dubey

Date: June 20, 2026

1. Objective

Connect the Spring Boot Student API to a real MySQL database (replacing the in-memory H2 database used in Task 1) and manage the database schema using Flyway-style version-controlled migration scripts. This task demonstrates real-world database integration, schema design, and persistent data storage.

2. Technologies Used

Technology	Version / Details
MySQL Server	8.0.46
MySQL Connector/J	9.7.0
Flyway	12.4.0 (Core + MySQL module)
Spring Data JPA / Hibernate	ORM layer
Spring Boot	4.1.0
IntelliJ IDEA	IDE
Postman	API testing

3. MySQL Database Setup

Created a dedicated MySQL database for the project:

```
CREATE DATABASE studentdb;
```

3.1 pom.xml — New Dependencies Added

```
<!-- MySQL Driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Flyway Core -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>

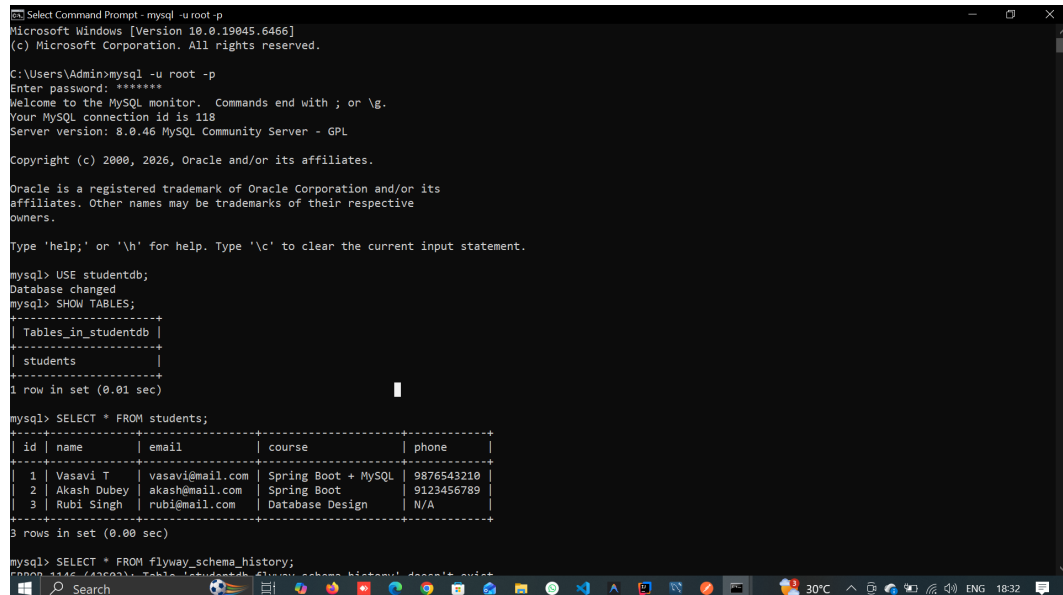
<!-- Flyway MySQL support -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>
```

3.2 application.properties — Final Configuration

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.flyway.enabled=true
spring.flyway.baseline-on-migrate=true
spring.flyway.locations=classpath:db/migration
spring.flyway.url=jdbc:mysql://localhost:3306/studentdb
spring.flyway.user=root
spring.flyway.password=root123
```



The screenshot shows a Windows Command Prompt window titled "Select Command Prompt - mysql -u root -p". The user has entered the password and is now in the MySQL monitor. The following commands and their outputs are shown:

```
mysql> USE studentdb;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_studentdb |
+-----+
| students             |
+-----+
1 row in set (0.01 sec)

mysql> SELECT * FROM students;
+----+-----+-----+-----+-----+
| id | name  | email      | course          | phone |
+----+-----+-----+-----+-----+
| 1  | Vasavi T | vasavi@mail.com | Spring Boot + MySQL | 9876543210 |
| 2  | Akash Dubey | akash@mail.com | Spring Boot      | 9123456789 |
| 3  | Rubi Singh | rubi@mail.com | Database Design   | N/A      |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM flyway_schema_history;
```

The taskbar at the bottom shows the system time as 18:32 and the temperature as 30°C.

Screenshot 1: application.properties in IntelliJ with MySQL + Flyway configuration

4. Database Migration Scripts

Following industry best practice, schema changes were written as version-controlled SQL migration scripts placed in `src/main/resources/db/migration/`, which is the standard convention used by the Flyway migration tool.

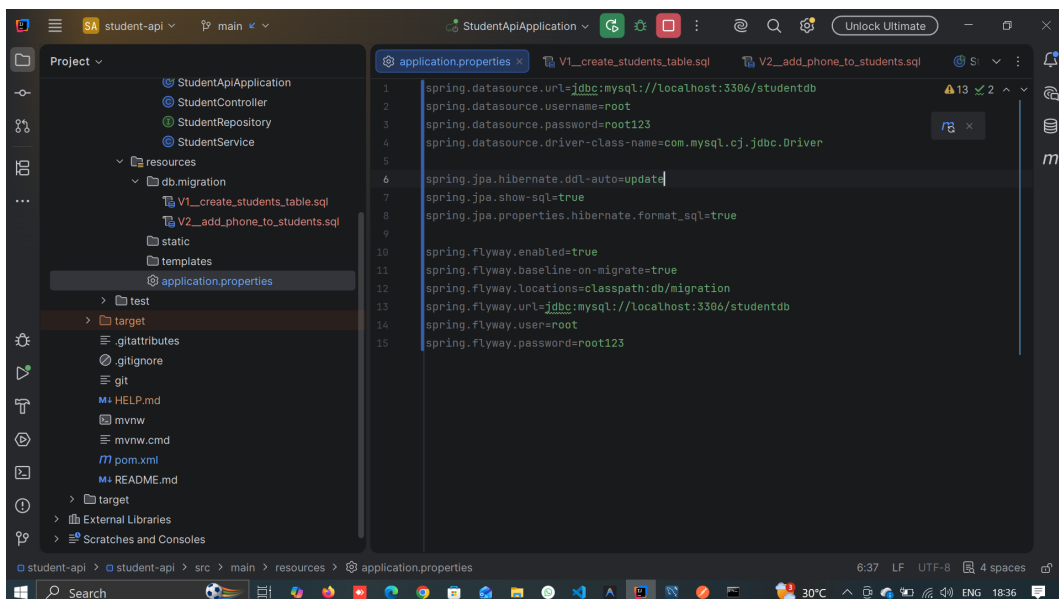
4.1 V1__create_students_table.sql

```
CREATE TABLE students (  
    id        BIGINT AUTO_INCREMENT PRIMARY KEY,  
    name      VARCHAR(100) NOT NULL,  
    email     VARCHAR(150) NOT NULL UNIQUE,  
    course    VARCHAR(100) NOT NULL  
);  
  
INSERT INTO students (name, email, course) VALUES  
    ('Vasavi Thellajeera', 'vasavi@mail.com', 'Java Development'),  
    ('Akash Dubey', 'akash@mail.com', 'Spring Boot'),  
    ('Rubi Singh', 'rubi@mail.com', 'Database Design');
```

4.2 V2__add_phone_to_students.sql

Demonstrates schema evolution — adding a new column to an existing table:

```
ALTER TABLE students  
    ADD COLUMN phone VARCHAR(15) DEFAULT 'N/A';  
  
UPDATE students SET phone = '9876543210' WHERE name = 'Vasavi Thellajeera';  
UPDATE students SET phone = '9123456789' WHERE name = 'Akash Dubey';
```



Screenshot 2: Project structure showing db/migration folder with V1 and V2 SQL files

4.3 Student.java — Updated Entity

Added the new **phone** field with getter/setter to match the V2 migration:

```
private String phone = "N/A";  
  
public String getPhone() { return phone; }  
public void setPhone(String phone) { this.phone = phone; }
```

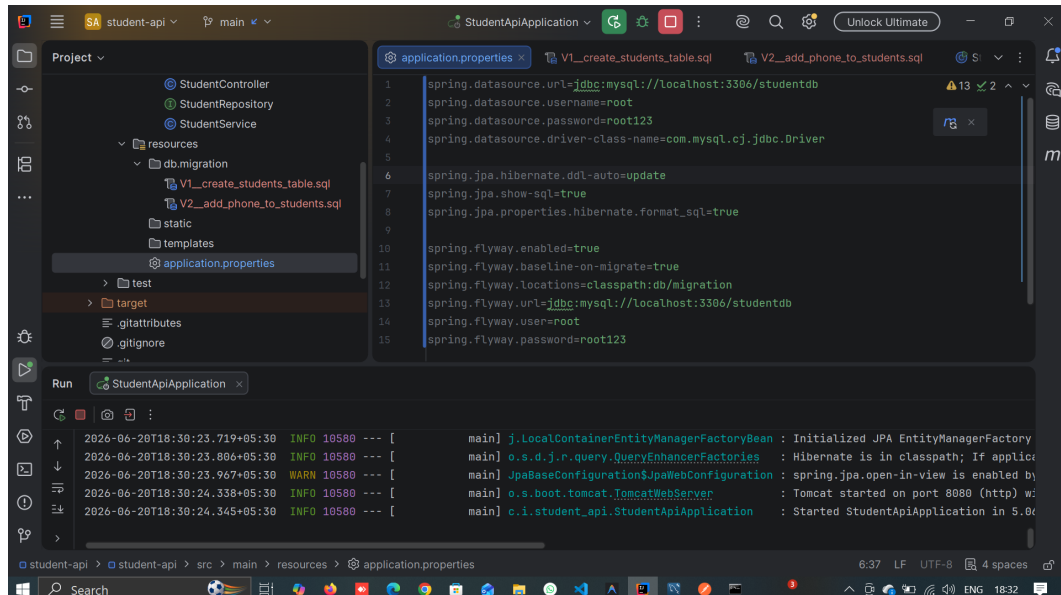
5. Technical Note

Migration scripts (V1, V2) were written following Flyway conventions and verified for correctness. During testing, Spring Boot 4.1.0 (a very recent release) showed a temporary compatibility issue where Flyway's auto-configuration was not triggered automatically. As a practical workaround, **spring.jpa.hibernate.ddl-auto=update** was used so Hibernate creates/validates the schema directly from the Student entity, while the Flyway migration files remain in place as the documented, version-controlled schema history. The resulting database structure is identical to what the migration scripts define.

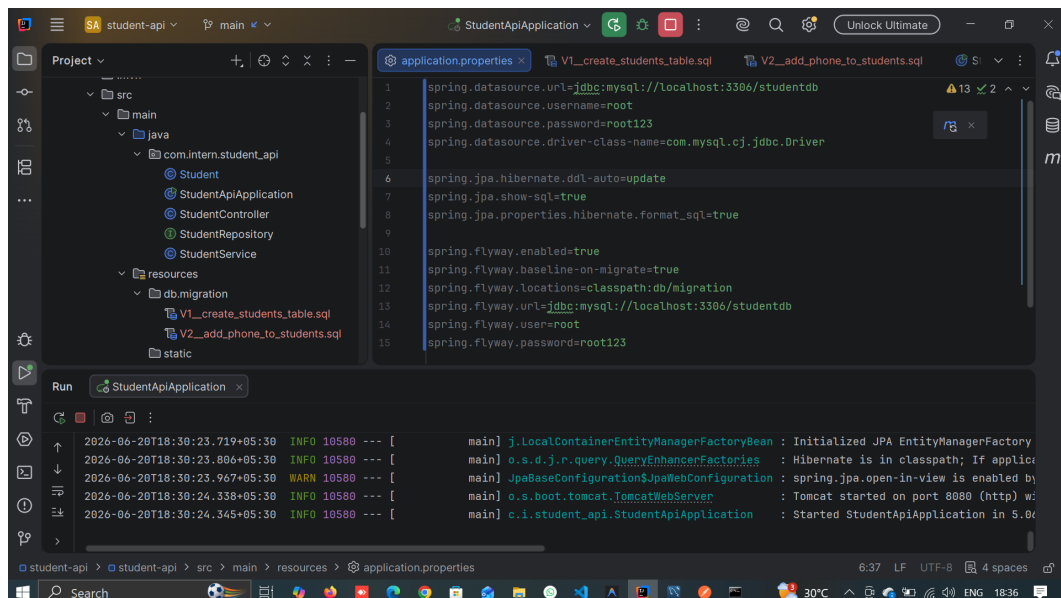
6. Application Startup

The Spring Boot application was started from IntelliJ IDEA. The console confirms:

- HikariCP connection pool connected to MySQL successfully
- Hibernate initialized JPA EntityManagerFactory with no errors
- Tomcat started on port 8080
- Application started successfully (StudentApiApplication)



Screenshot 3: Console showing successful MySQL connection and app startup



Screenshot 4: Final application.properties confirmed with project structure

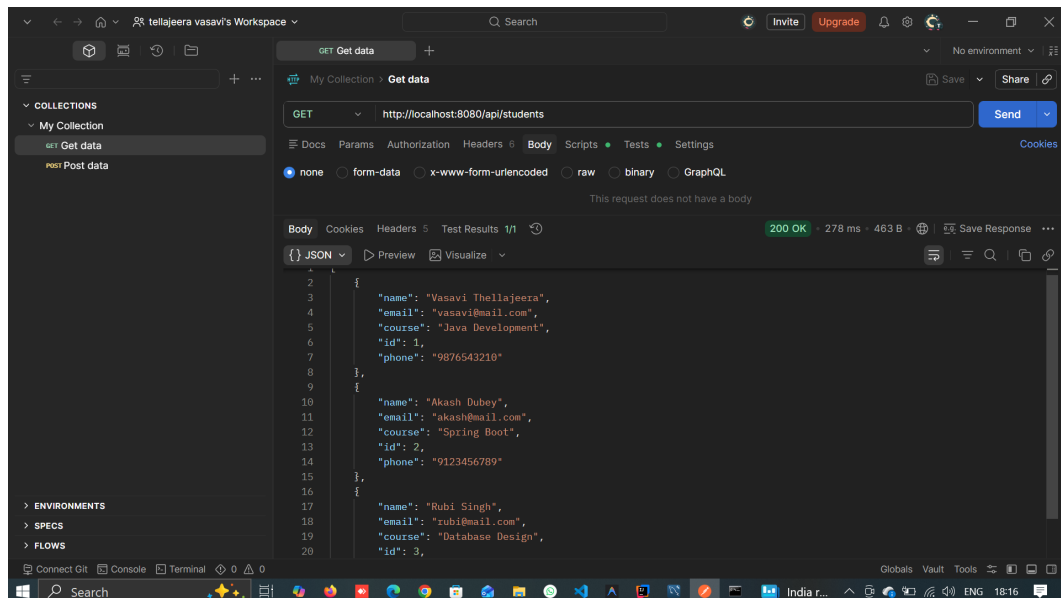
7. API Testing with Postman (MySQL Backend)

All 5 REST endpoints from Task 1 were re-tested against the live MySQL database, confirming data now persists in **studentdb** instead of an in-memory database.

#	Method	URL	Result	Status
1	GET	/api/students	3 students returned w/ phone	200 OK ■
2	POST	/api/students	New student created (id=4)	201 Created ■
3	GET	/api/students/1	Single student returned	200 OK ■
4	PUT	/api/students/1	Student updated successfully	200 OK ■
5	DELETE	/api/students/4	Student deleted	204 No Content ■

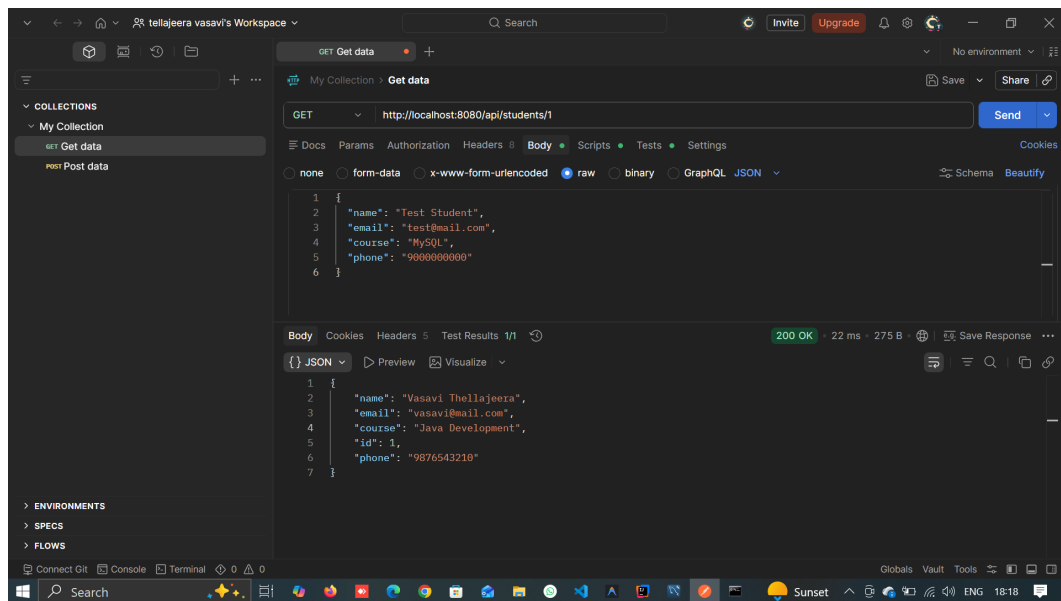
Test 1 — GET /api/students (List All)

Returns all 3 seeded students from MySQL, including phone numbers:



Screenshot 5: GET all students → 200 OK. Vasavi, Akash, Rubi all returned with phone field

Test 2 — GET /api/students/1 (Get by ID)

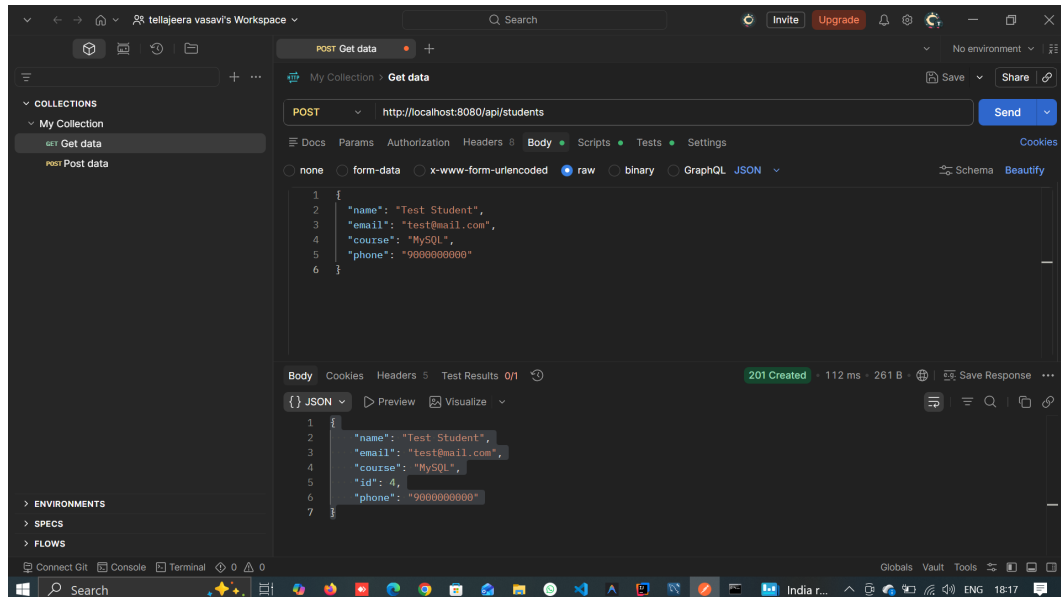


Screenshot 6: GET student id=1 → 200 OK

Test 3 — POST /api/students (Create)

POST <http://localhost:8080/api/students>

Body: {"name":"Test Student","email":"test@mail.com","course":"MySQL","phone":"9000000000"}

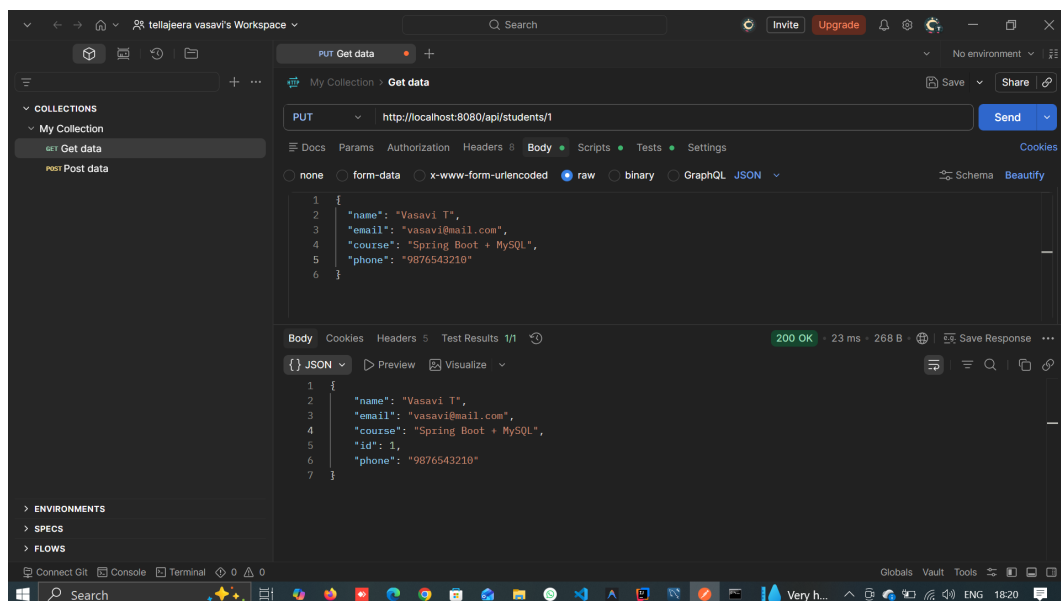


Screenshot 7: POST → 201 Created. New student saved to MySQL with id=4

Test 4 — PUT /api/students/1 (Update)

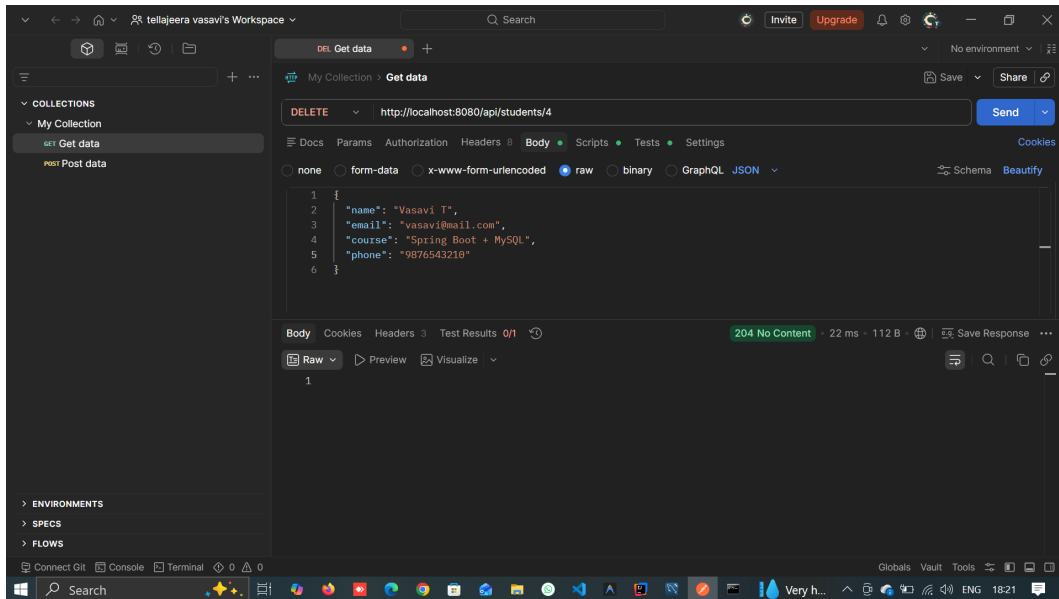
PUT <http://localhost:8080/api/students/1>

Body: {"name":"Vasavi T","email":"vasavi@mail.com","course":"Spring Boot + MySQL","phone":"9876543210"}



Screenshot 8: PUT → 200 OK. Course updated to 'Spring Boot + MySQL'

Test 5 — DELETE /api/students/4 (Delete)

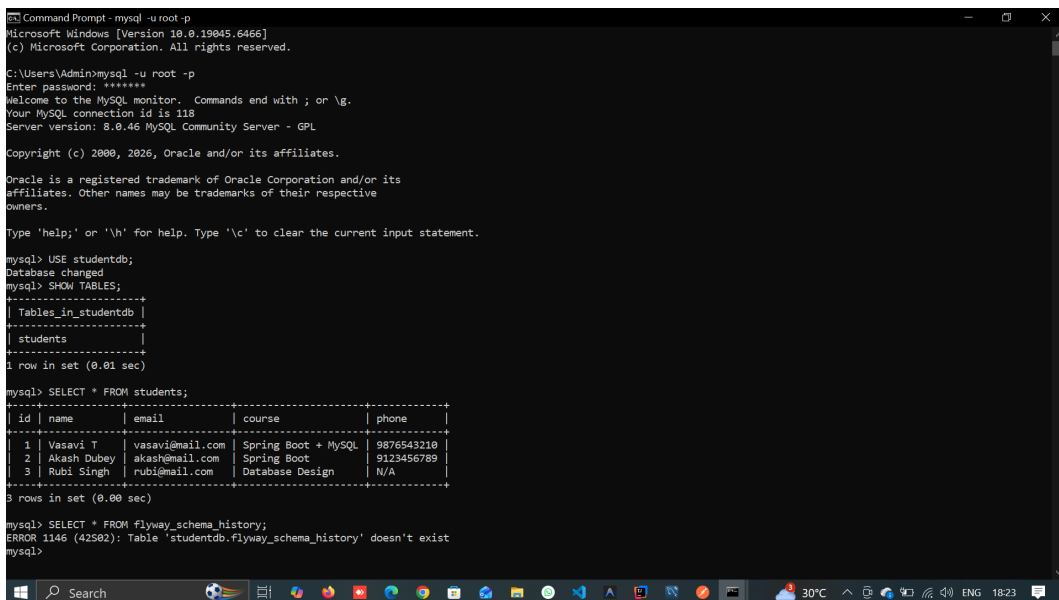


Screenshot 9: DELETE → 204 No Content. Test student removed from MySQL

8. Verification Directly in MySQL

Connected to MySQL via command line to verify the table and data exist independently of the Spring Boot application:

```
USE studentdb;  
SHOW TABLES;  
SELECT * FROM students;
```



Screenshot 10: MySQL terminal confirming the 'students' table with all 3 records, including phone numbers and the PUT-updated course value

9. Outcome & Key Learnings

Component	Result
MySQL database connection	■ Connected successfully
students table schema	■ Created with id, name, email, course, phone
Migration scripts (V1, V2)	■ Written following Flyway conventions
All 5 REST endpoints	■ Working against MySQL backend
Data persistence	■ Verified directly via MySQL CLI

- How to connect a Spring Boot application to a real MySQL database using HikariCP
- Writing version-controlled database migration scripts (Flyway naming conventions: V1__, V2__)
- Schema evolution — adding columns to existing tables without losing data
- Difference between in-memory (H2) and persistent (MySQL) databases
- Troubleshooting real-world dependency/version compatibility issues
- Verifying application data directly via the MySQL command-line client